

Omitted Fiat–Shamir Binding Enables Universal Forgery

A Public-Key-Only Break of All 14 Submitted Chinith Parameter Sets

Martin Feussner

Selmer Center, University of Bergen

`martin.feussner@uib.no`

25 September 2026

Provenance and review status. The attack was independently found by an AI system, OpenAI Codex using Daybreak Blue at maximum reasoning effort, without a human first proposing this specific attack. The derivation, implementation, experiments, and initial manuscript were also produced by the AI. At the date above, no human cryptanalyst has independently verified the argument, experiments, or novelty assessment. This is a preliminary disclosure for human review and reproduction.

Abstract

Chinith is a family of VOLE-in-the-head signatures submitted to the Chinese Next-generation Commercial Cryptographic Algorithms competition. Its QuickSilver proof reduces satisfaction of the public one-way-function relation to a compressed constant term $\tilde{a}_0$. The specification binds that value into the final Fiat–Shamir challenge together with $\tilde{a}_1, \dots, \tilde{a}_{d-1}$. Both submitted implementation trees instead hash only $\tilde{a}_1, \dots, \tilde{a}_{d-1}$. The signer computes $\tilde{a}_0$ and discards it. The verifier reconstructs $\tilde{a}_0$ from the public relation and proof transcript, then also discards it. Consequently, verification never tests whether the committed witness satisfies the relation represented by the public key.

This yields a universal forgery requiring no signature queries and no secret information. Given any public key and chosen message, an attacker selects an arbitrary witness and arbitrary seed material, executes the public prover operations, derives the defective final challenge, and emits an ordinary signature. Every remaining commitment, consistency, grinding, and message binding check is honestly satisfied; the sole missing check is the one that would reject the false witness.

We implemented this attack against the submitted portable code. The untouched reference verifiers accepted public-key-only forgeries for every one of the 14 SM4th, uBlockith, and Vistrutith parameter sets. Forging took 0.011–2.660 single-threaded CPU-seconds. Complete processes, including key generation solely to obtain a legitimate target public key and two verifier calls, took 0.07–7.92 seconds and at most 55 MB of resident memory on a 3.2 GHz Intel Core i5-6500. Changing one bit of the target message caused every forged signature to be rejected. A static audit finds the same omission in all 14 reference and all 14 optimized implementations. The written specification contains the missing binding, so the result is a complete break of the submitted implementations rather than an attack on the correctly specified transcript.

1 Introduction

Chinith comprises the SM4th, uBlockith, and Vistrutith signature families. Each proves knowledge of a block-cipher key or input satisfying a public one-way-function relation. The submission

builds this proof from VOLE-in-the-head commitments and a QuickSilver-style arithmetic proof, then applies the Fiat–Shamir transform to obtain a non-interactive signature [1, 3, 2, 4].

The final Fiat–Shamir challenge has two jobs. It selects the VOLE openings, and it binds the compressed QuickSilver proof into the transcript. For degree- d constraints, the prover obtains field elements $\tilde{a}_0, \dots, \tilde{a}_{d-1}$. The verifier reconstructs $\tilde{a}_0$ from its own evaluation of the public relation. It then hashes that reconstructed value together with the transmitted higher coefficients and checks that the result equals the opening challenge. This fixed-point check is what makes a false witness fail.

The submitted code omits exactly the reconstructed term. On both sides, the final hash begins at the serialized address of $\tilde{a}_1$ and consumes $(d - 1)$ field elements. Thus it hashes $\tilde{a}_1, \dots, \tilde{a}_{d-1}$ and never $\tilde{a}_0$. This is consistent between signer and verifier, so known answer tests generated by the same code do not reveal the error.

The omission removes the public-key relation from acceptance. An attacker can run the prover with the all-zero witness, or any other bit string of the required length, while retaining the victim’s public key as the public input to the constraint system. The VOLE layer correctly commits to that false witness. The constraint layer computes evidence that it is false, but that evidence appears only in $\tilde{a}_0$, which acceptance ignores.

Our contributions are:

- We identify the omitted constant proof term in all 28 submitted implementation directories: 14 reference and 14 optimized variants.
- We give a public-key-only universal forgery. It uses no signing oracle, no recovered key, no malformed input, and no rare event.
- We prove acceptance by separating the still-valid VOLE transcript checks from the discarded QuickSilver relation check.
- We reproduce the forgery end to end against the untouched reference verifier for all 14 parameter sets, including the degree-2 and degree-3 proof paths and all three block-cipher families.
- We describe a direct repair and regression tests that distinguish a corrected implementation from the submitted one.

2 The relation check in Chinit

We summarize only the elements needed for the attack. Let the public key be $\mathbf{pk} = (x, y)$. Depending on the variant, the claimed relation is either

$$y = E_k(x) \tag{1}$$

or the Even–Mansour relation

$$y = E_x(k) + k. \tag{2}$$

Here and below addition is over the relevant characteristic-two space. The signer extends the secret k into a bit witness $w \in \{0, 1\}^\ell$ that contains the internal values needed to check the selected block cipher.

2.1 VOLE commitment and masked witness

The prover obtains a VOLE correlation consisting of a bit vector u , masks $v_i \in \mathbb{F}_{2^{\lambda_F}}$, verifier values q_i , and a global challenge $\Delta \in \mathbb{F}_{2^{\lambda_F}}$ satisfying

$$q_i = u_i \Delta + v_i. \tag{3}$$

It sends the masked witness

$$D = w + u. \quad (4)$$

After reconstruction, the verifier can therefore treat $q_i + D_i\Delta$ as a commitment to w_i with mask v_i .

The first two Fiat–Shamir challenges bind the message, public key, VOLE commitment, consistency data, and D . None of those steps requires w to satisfy Equation (1) or (2); they establish only that the later proof refers to the same committed bit string.

2.2 Compressed QuickSilver check

Chinith evaluates the public constraint system on the VOLE-committed witness. For a degree- d relation, each compressed constraint has the polynomial form

$$\rho(X) = a_0 + a_1X + \cdots + a_{d-1}X^{d-1} + zX^d, \quad (5)$$

where the leading coefficient z vanishes when the constraint is satisfied. A universal hash seeded by the second challenge batches all constraints and masking terms into

$$\tilde{a}_0, \tilde{a}_1, \dots, \tilde{a}_{d-1} \in \mathbb{F}_{2^{\lambda_F}}.$$

The verifier evaluates its committed view at $\Delta = \text{ToField}(\text{chall}_3)$ and derives a value $\tilde{q}$. In the submission’s characteristic-two notation it reconstructs

$$\tilde{a}_0^* = \tilde{q} + \sum_{i=1}^{d-1} \tilde{a}_i \Delta^i. \quad (6)$$

Using Equation (5), the reconstructed constant term can be written as

$$\tilde{a}_0^* = \tilde{a}_0 + \tilde{z} \Delta^d, \quad (7)$$

where $\tilde{z}$ is the compressed leading coefficient of the batched constraint polynomial. For a valid committed witness, $\tilde{z} = 0$ and therefore $\tilde{a}_0^* = \tilde{a}_0$. For an invalid witness, the discrepancy in Equation (7) is exactly the value that the final Fiat–Shamir binding is supposed to expose; equality fails except with the intended soundness probability.

2.3 The specified final transcript

The specification derives the final challenge inside its grinding loop as

$$\text{chall}_3 = \text{Hthree}(\text{chall}_2 \parallel \tilde{a}_0 \parallel \tilde{a}_1 \parallel \cdots \parallel \tilde{a}_{d-1} \parallel \text{ctr}). \quad (8)$$

The signature transmits $\tilde{a}_1, \dots, \tilde{a}_{d-1}$ but omits $\tilde{a}_0$, because the verifier reconstructs it with Equation (6). Correct verification substitutes $\tilde{a}_0^*$ into Equation (8) and accepts only when the resulting challenge equals the transmitted chall_3 .

This creates the necessary dependency cycle for soundness. The final challenge determines Δ and hence the verifier’s reconstructed constant term, while the constant term is itself included in the hash that must produce the final challenge. An honest proof satisfies the cycle. A prover using a false witness cannot simply derive the challenge before learning the value at which its inconsistent constraint polynomial will be checked.

3 The submitted omission

The implementation allocates and computes $\tilde{a}_0$ during signing. It serializes only the higher terms, as the compact signature format requires. When initializing the final hash, however, it supplies a pointer to the first serialized term $\tilde{a}_1$ and hashes exactly $d - 1$ field elements. Its effective challenge is

$$\text{chall}_3 = H_{\text{three}}(\text{chall}_2 \parallel \tilde{a}_1 \parallel \dots \parallel \tilde{a}_{d-1} \parallel \text{ctr}). \quad (9)$$

The locally computed $\tilde{a}_0$ is freed without affecting any serialized byte or challenge.

Verification makes the matching error. It evaluates the block-cipher constraints and computes $\tilde{a}_0^*$ using Equation (6). It then initializes the final hash at the signature's $\tilde{a}_1$ field and consumes the same $d - 1$ elements. The reconstructed value is freed without being compared or hashed.

The code pattern appears in every parameter directory. It is shared across the four standard SM4th variants, four SM4th-EM variants, two standard uBlockith variants, two uBlockith-EM variants, and two Vistrutith variants. The same signer and verifier pattern occurs in both the portable reference and optimized implementation trees. The degree does not protect any variant: the degree-2 path hashes only $\tilde{a}_1$, while the degree-3 paths hash $\tilde{a}_1 \parallel \tilde{a}_2$.

Equation (9) contains no result of the verifier's public relation evaluation. The block-cipher constraint code still runs, but its output has no path to the acceptance decision.

3.1 Code-level evidence

The relevant source-level behavior can be summarized by the following normalized pseudocode. The notation is schematic: it records the data flow observed in the submitted signer and verifier without depending on local variable names or memory-layout details.

Specification:

```
chall3 = H3(chall2 || a0 || a1 || ... || a[d-1] || ctr)
```

Submitted signer:

```
(a0, a1, ..., a[d-1]) = OWF_Prove(...)
chall3 = H3(chall2 || a1 || ... || a[d-1] || ctr)
```

Submitted verifier:

```
a0_star = ReconstructRelationCheck(...)
chall3_star = H3(chall2 || a1 || ... || a[d-1] || ctr)
accept only checks chall3_star == chall3
```

Thus the signer computes the constant relation-check term and the verifier reconstructs it, but neither value is absorbed into the final challenge. The same omission is present in the portable reference and optimized trees. An independent source audit should record the exact file names, function names, and line or commit locations for these three operations when reproducing the result; this report does not invent source identifiers that have not yet been independently checked.

4 Universal forgery

The attack uses the ordinary public algorithms. The arbitrary value denoted k^* below is used only as input material for deriving commitment randomness; it is not asserted to be a preimage of the public key.

Input. A public key $\text{pk} = (x, y)$ and a chosen target message M .

Forgery algorithm.

1. Choose any bit string $w^* \in \{0, 1\}^\ell$, any string k^* of the block-cipher key length, and any salt ρ . All-zero w^* and k^* suffice.
2. Compute the message digest from pk and M . Use k^* , the digest, and ρ in the submitted randomness derivation to obtain a root seed and IV.
3. Run the ordinary VOLE commitment. Derive chall_1 , compute the VOLE consistency response, and set $D = w^* + u$. Derive chall_2 exactly as the signer does.
4. Run the ordinary, publicly specified OWF prover routine on w^* and the victim public key. It returns $(\tilde{a}_0, \dots, \tilde{a}_{d-1})$. Retain the serialized values $\tilde{a}_1, \dots, \tilde{a}_{d-1}$; the value $\tilde{a}_0$ is irrelevant to the submitted verifier.
5. Perform the normal counter search using Equation (9). Decode the resulting challenge and produce a valid BAVC opening, retrying the counter exactly as in ordinary signing.
6. Return the normal signature encoding of the commitment corrections, consistency response, D , higher proof coefficients, opening, chall_3 , IV preimage, and counter.

Every operation above is public and has essentially the cost of one signing operation. No transcript from a legitimate signer is used.

Theorem 1 (Acceptance of the forgery). *For every honestly generated Chinith public key pk , every target message M , and every chosen witness w^* of the required format, whenever the ordinary grinding-and-opening procedure succeeds, the submitted verifier accepts the resulting signature independently of whether w^* satisfies the public OWF relation.*

Proof. The commitment and opening are generated honestly, so VOLE reconstruction succeeds. Both parties recompute the same first challenge because they use the same public key, message digest, commitment, correction values, and IV. Equation (4) and the valid VOLE correlation imply that the verifier’s consistency computation represents the same chosen witness w^* . It therefore reconstructs the same second challenge.

The verifier next evaluates the public OWF constraints on its committed view and obtains some $\tilde{a}_0^*$. This value need not equal the prover’s $\tilde{a}_0$ because w^* may be false. Under the submitted acceptance rule, neither value is used. The only proof coefficients absorbed by the final hash are the transmitted $\tilde{a}_1, \dots, \tilde{a}_{d-1}$. The verifier hashes those same bytes with the already matching chall_2 and counter. It therefore recomputes exactly the attacker’s chall_3 . The grinding predicate and BAVC opening were selected for that challenge, so all remaining checks also pass. $\square$

Remark 2. *The target message remains bound to the forged signature through the earlier challenges. This does not impede forgery because the attacker chooses the message before producing the transcript. It does predict that changing the message after forging should cause rejection, which our experiments confirm.*

5 End-to-end evaluation

We evaluated the attack on the official Chinith submission archive with SHA-256 digest

4d31ba3fe8718f59901b7efdf01b4179912ef8e04f58b4b6dbba84186f002283.

The experiments used the portable reference implementation, GCC 13.3.0, and the submission’s release-style optimization and assertion settings. The machine was a four-core Intel Core i5-6500 at 3.2 GHz with 16 GiB of RAM, running 64-bit Linux. Each forgery itself ran on one thread.

For each parameter set, we first generated a valid key pair solely to obtain a legitimate target public key. We then overwrote the secret key and the inputs from which it was generated. From that point onward the attack received only the public key, a 37-byte target message, a chosen salt, an all-zero fake key, and an all-zero witness of the parameter-prescribed length. It called the public prover operations and passed the result to the untouched submitted verification API. We separately evaluated the one-way function under the all-zero fake key and confirmed that its output differed from the target public-key output in every run.

For a negative control, we flipped the first bit of the message and invoked the same verifier on the same forged signature. In all 14 cases, verification returned success for the target and failure for the altered message. Table 1 gives the serialized signature size, CPU time spent in the forgery, whole-process elapsed time, and peak resident memory. Whole-process time includes target key generation, forgery, successful verification, and the altered-message verification.

Table 1: Public-key-only forgeries against all submitted reference variants. Every target signature was accepted and every one-bit message alteration was rejected.

Variant	d	Signature (bytes)	Forge CPU (seconds)	Total wall (seconds)	Peak RSS (MiB)
SM4th-d3-128f-loose	3	6,724	0.043	0.12	3.1
SM4th-d3-128f-tight	3	11,864	0.067	0.15	3.8
SM4th-d3-128s-loose	3	5,056	0.123	0.34	10.3
SM4th-d3-128s-tight	3	9,176	0.145	0.78	15.1
SM4th-EM-d2-128f-loose	2	4,932	0.011	0.07	2.5
SM4th-EM-d2-128f-tight	2	9,450	0.012	0.07	3.1
SM4th-EM-d2-128s-loose	2	3,818	0.071	0.50	10.6
SM4th-EM-d2-128s-tight	2	7,556	0.081	0.71	11.6
uBlockith-d3-256f	3	31,556	0.066	0.25	12.8
uBlockith-d3-256s	3	24,144	0.305	1.63	16.6
uBlockith-EM-d3-256f	3	25,028	0.048	0.31	3.7
uBlockith-EM-d3-256s	3	19,056	0.260	2.26	16.0
Vistrutith-d3-512f	3	106,788	0.552	2.84	8.5
Vistrutith-d3-512s	3	83,004	2.660	7.92	54.6

The measurements show a stronger result than laptop-scale key recovery. The adversary needs zero signing queries, less than three CPU-seconds for the largest forgery, and memory measured in tens of megabytes. Signature sizes are identical to ordinary signatures for their respective parameter sets.

As a diagnostic before constructing the public-key-only attack, we also used the SM4th-d3-128s-loose prover with an honestly generated public key and modified each of the 226 encoded witness-byte positions individually. All 226 resulting proofs were accepted. This experiment is diagnostic only: the modified byte strings need not all correspond to semantically valid alternative witnesses. The all-zero-witness experiments in Table 1 are the stronger end-to-end result because they demonstrate acceptance for a single explicit false witness whose one-way-function output was separately checked not to match the target public key.

6 Repair and validation

The direct repair is to implement Equation (8). During signing, the final-hash state must absorb the locally computed $\tilde{a}_0$ and then every serialized field element $\tilde{a}_1, \dots, \tilde{a}_{d-1}$ before absorbing the counter. During verification, it must absorb the reconstructed $\tilde{a}_0^*$ followed by the same serialized higher coefficients. Changing only the starting pointer is insufficient because $\tilde{a}_0$ is stored separately from the signature buffer.

The repaired challenge changes every signature, so submitted known-answer vectors will need regeneration. Keys and the public signature layout need not change: $\tilde{a}_0$ remains reconstructible and therefore need not be transmitted.

At minimum, validation should include the following negative tests:

- generate the prover transcript with an arbitrary witness under an unrelated valid public key and require rejection;
- change every witness byte in turn, regenerate the proof, and require rejection except when the modified value independently satisfies the relation;
- compare the exact byte sequence absorbed into the signer’s and verifier’s final hashes and confirm that it begins with the respective $\tilde{a}_0$ value;
- run cross-implementation known-answer tests generated from a corrected transcript definition rather than accepting agreement between two implementations that share the same helper.

Including $\tilde{a}_0$ restores the intended QuickSilver relation check, but the repaired implementation and its security claims should be reviewed in full. This report does not claim that the correctly specified construction has a comparable forgery.

7 Scope and disclosure status

The result applies to the Chinith archive identified in Section 5. It covers every submitted parameter set and both implementation trees at the source level; the end-to-end measurements use the portable reference tree. It does not depend on a crash, undefined input, uninitialized memory, weak entropy, timing leakage, or a signing oracle. The signature is produced for an attacker-chosen message and is accepted by the ordinary verifier for the victim public key.

The mathematical algorithms in the submitted specification explicitly place $\tilde{a}_0$ in the final hash. Accordingly, our claim is a universal forgery against the submitted software realizations and their generated test vectors. We do not claim a break of an implementation that follows the specified transcript.

The attack was independently found by OpenAI Codex using Daybreak Blue at maximum reasoning effort, without a human first proposing this specific attack. Human verification, independent reproduction, and novelty review remain pending.

8 Conclusion

Chinith’s submitted signer and verifier agree on a final Fiat–Shamir transcript that excludes the one reconstructed field element carrying the QuickSilver relation check. That agreement makes ordinary signatures verify and self-generated known-answer tests pass, while allowing any committed witness to stand in for the secret key. The resulting attack is a zero-query, public-key-only universal forgery across all 14 submitted parameter sets, with sub-second forging time for 13 variants and 2.660 seconds for the largest measured variant.

References

- [1] Chinith submission team. *Chinith: Algorithm Specifications*. NGCC submission package, 2026.
- [2] K. Yang, P. Sarkar, C. Weng, and X. Wang. QuickSilver: Efficient and Affordable Zero-Knowledge Proofs for Circuits and Polynomials over Any Field. In *ACM CCS*, pages 2986–3001, 2021.
- [3] C. Baum, L. Braun, C. D. de Saint Guilhem, M. Klooß, E. Orsini, L. Roy, and P. Scholl. Publicly Verifiable Zero-Knowledge and Post-Quantum Signatures from VOLE-in-the-Head. In *CRYPTO*, pages 581–615, 2023.

- [4] A. Fiat and A. Shamir. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *CRYPTO*, pages 186–194, 1986.
- [5] C. Baum, W. Beullens, L. Braun, et al. *FAEST v2: Algorithm Specifications, Version 2.0*. 2025.